



AMRITA
VISHWA VIDYAPEETHAM

School of Engineering
Chennai Campus

Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

NAME: AGNEAY B NAIR
REG NO: CH.SC.U4CSE24102

WORKSHEET



WORKSHEET- 2

EX.NO : 2

DATE : 18/06/2026

TITLE : STUDY AND VISUALIZATION OF SINGLE-QUBIT STATES USING THE BLOCH SPHERE

AIM:

To study the representation of single-qubit quantum states using the Bloch Sphere and visualize the effect of various quantum gates on qubit states using the Qiskit framework. The objective is to understand quantum state representation, superposition, and state transformations through Bloch Sphere visualization.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit Quantum Computing Framework
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

THEORY:

Introduction to the Bloch Sphere:

The Bloch Sphere is a geometrical representation of the state space of a single qubit. It provides an intuitive way to visualize and analyze quantum states and their transformations. Every pure state of a single qubit can be represented as a point on the surface of the Bloch Sphere.

Single-Qubit Quantum States:

Unlike a classical bit, which can exist only in the states 0 or 1, a qubit can exist in a superposition of both states. The general state of a qubit is represented as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$



where α and β are complex probability amplitudes satisfying:

$$|\alpha|^2 + |\beta|^2 = 1$$

Bloch Sphere Representation:

A single-qubit state can be represented on the Bloch Sphere using two parameters, θ and ϕ , as:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{(i\phi)\sin\left(\frac{\theta}{2}\right)}|1\rangle$$

where:

- θ represents the polar angle.
- ϕ represents the azimuthal angle.

These angles determine the position of the qubit on the Bloch Sphere.

Computational Basis States:

The computational basis states are represented by the poles of the Bloch Sphere:

- North Pole $\rightarrow |0\rangle$
- South Pole $\rightarrow |1\rangle$

These states correspond to the classical binary states.

Superposition States:

States located on the equator of the Bloch Sphere represent equal superpositions of $|0\rangle$ and $|1\rangle$. The Hadamard gate is commonly used to generate such superposition states.

Quantum Gates and State Rotation:

Quantum gates can be interpreted as rotations of the qubit state vector on the Bloch Sphere.

- Pauli-X Gate: Rotation about the X-axis.
- Pauli-Y Gate: Rotation about the Y-axis.
- Pauli-Z Gate: Rotation about the Z-axis.
- Hadamard Gate: Creates superposition states.

These gates change the position of the qubit state on the Bloch Sphere.

Bloch Sphere Visualization Using Qiskit:



Qiskit provides visualization tools to represent quantum states on the Bloch Sphere. By applying different quantum gates and generating state vectors, the resulting qubit states can be visualized and analyzed effectively.

Significance of the Bloch Sphere:

The Bloch Sphere is widely used for understanding quantum state evolution, quantum gate operations, superposition, and quantum algorithms. It serves as a fundamental visualization tool in quantum computing and quantum information processing.

PROCEDURE:

1. Open Jupyter Notebook or Visual Studio Code.
2. Import the required Qiskit libraries and visualization modules.
3. Create a single-qubit quantum circuit.
4. Initialize the qubit in the computational basis state $|0\rangle$.
5. Generate the state vector of the qubit.
6. Visualize the quantum state using the Bloch Sphere.
7. Apply quantum gates such as X, Y, Z, and H to the qubit.
8. Observe the changes in the qubit state on the Bloch Sphere.
9. Compare the Bloch Sphere representations obtained for different quantum gates.
10. Analyze and verify the resulting quantum states.

PROGRAM:

Program 1: Create a quantum circuit with a single qubit in the $|0\rangle$ state and visualize it on the Bloch Sphere.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

# Single-qubit circuit, qubit initialised by default in  $|0\rangle$ 
qc = QuantumCircuit(1)
print(qc.draw())

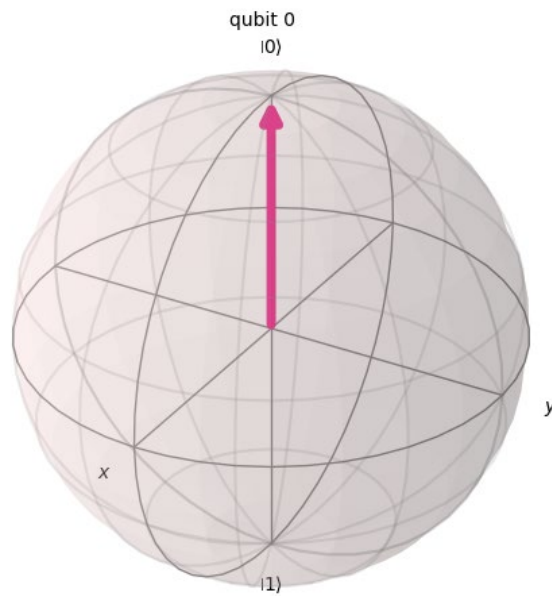
state = Statevector.from_instruction(qc)
print("Statevector:", state)

plot_bloch_multivector(state, title="Program 1:  $|0\rangle$  state")
```

OUTPUT:



Program 1: $|0\rangle$ state



Program 2: Apply the Pauli-X gate to a single qubit and visualize the resulting $|1\rangle$ state on the Bloch Sphere.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

qc = QuantumCircuit(1)
qc.x(0)                      # Pauli-X (NOT) gate:  $|0\rangle \rightarrow |1\rangle$ 
print(qc.draw())

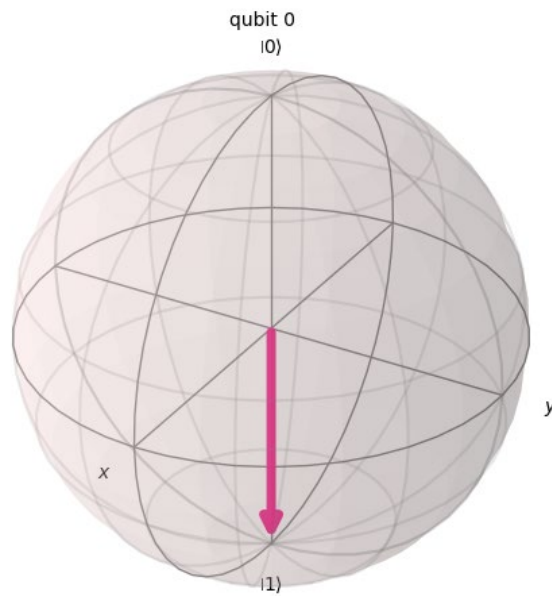
state = Statevector.from_instruction(qc)
print("Statevector:", state)

plot_bloch_multivector(state, title="Program 2: X gate  $\rightarrow |1\rangle$ ")
```

OUTPUT:



Program 2: X gate -> $|1\rangle$



Program 3: Create a quantum circuit using the Hadamard gate and visualize the superposition state on the Bloch Sphere.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

qc = QuantumCircuit(1)
qc.h(0)                                # Hadamard gate creates equal superposition
print(qc.draw())

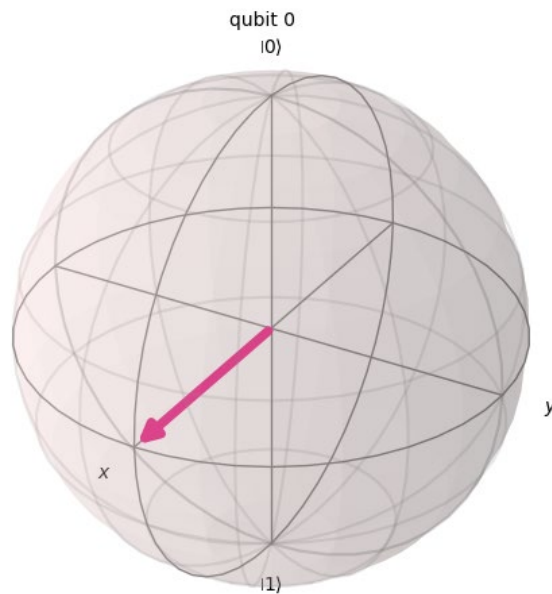
state = Statevector.from_instruction(qc)
print("Statevector:", state)

plot_bloch_multivector(state, title="Program 3: H gate -> superposition")
```

OUTPUT:



Program 3: H gate -> superposition $|+\rangle$



Program 4: Apply the Pauli-Y gate to a single qubit and observe the state transformation using the Bloch Sphere.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

qc = QuantumCircuit(1)
qc.y(0)                                # Pauli-Y gate:  $|0\rangle \rightarrow i|1\rangle$ 
print(qc.draw())

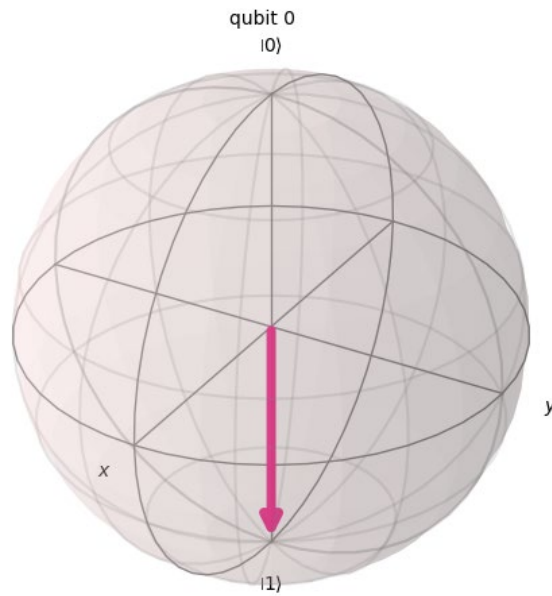
state = Statevector.from_instruction(qc)
print("Statevector:", state)

plot_bloch_multivector(state, title="Program 4: Y gate")
```

OUTPUT:



Program 4: Y gate



Program 5: Apply the Pauli-Z gate to a single qubit and visualize the resulting state on the Bloch Sphere.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

qc = QuantumCircuit(1)
qc.z(0)                                # Pauli-Z gate:  $|0\rangle \rightarrow |0\rangle$  (acts on  $|1\rangle$  component)
print(qc.draw())

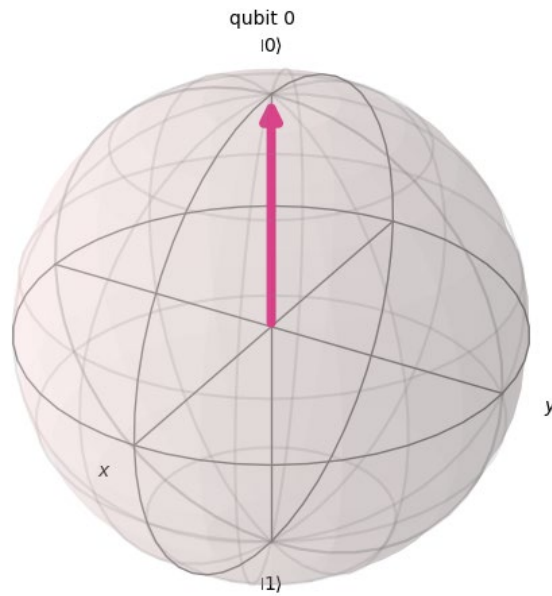
state = Statevector.from_instruction(qc)
print("Statevector:", state)

plot_bloch_multivector(state, title="Program 5: Z gate")
```

OUTPUT:



Program 5: Z gate



Program 6: Create and visualize the Bloch Sphere representation of the $|+\rangle$ state using the Hadamard gate.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

qc = QuantumCircuit(1)
qc.h(0)                                #  $|0\rangle \xrightarrow{H} |+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ 
print(qc.draw())

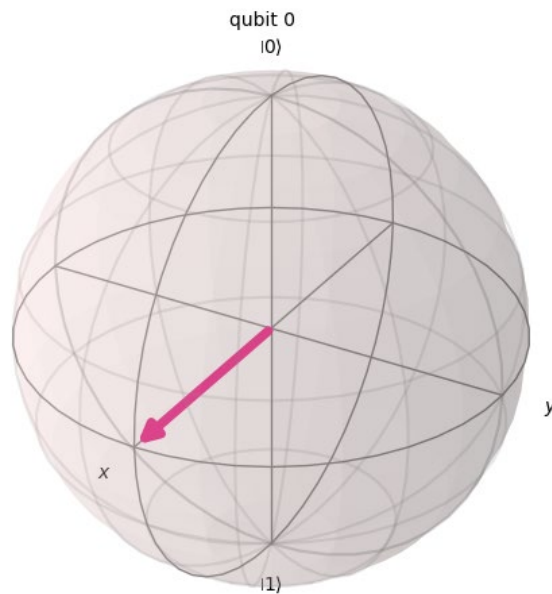
state = Statevector.from_instruction(qc)
print("Statevector:", state)

plot_bloch_multivector(state, title="Program 6:  $|+\rangle$  state")
```

OUTPUT:



Program 6: $|+\rangle$ state



Program 7: Create and visualize the Bloch Sphere representation of the $|-\rangle$ state using the Hadamard and Pauli-Z gates.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

qc = QuantumCircuit(1)
qc.h(0)
qc.z(0)                                     #  $|\theta\rangle \xrightarrow{H} |+\rangle \xrightarrow{Z} |-\rangle = (|\theta\rangle - |1\rangle)/\sqrt{2}$ 
print(qc.draw())

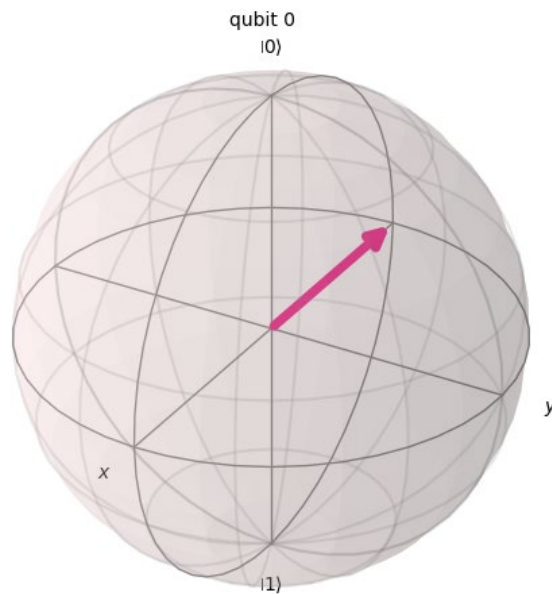
state = Statevector.from_instruction(qc)
print("Statevector:", state)

plot_bloch_multivector(state, title="Program 7:  $|-\rangle$  state")
```

OUTPUT:



Program 7: $|-\rangle$ state



Program 8: Apply a sequence of quantum gates (H followed by X) and observe the resulting state on the Bloch Sphere.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

qc = QuantumCircuit(1)
qc.h(0)                #  $|0\rangle \rightarrow |+\rangle$ 
qc.x(0)                #  $X|+\rangle = |+\rangle$  (X leaves  $|+\rangle$  unchanged)
print(qc.draw())

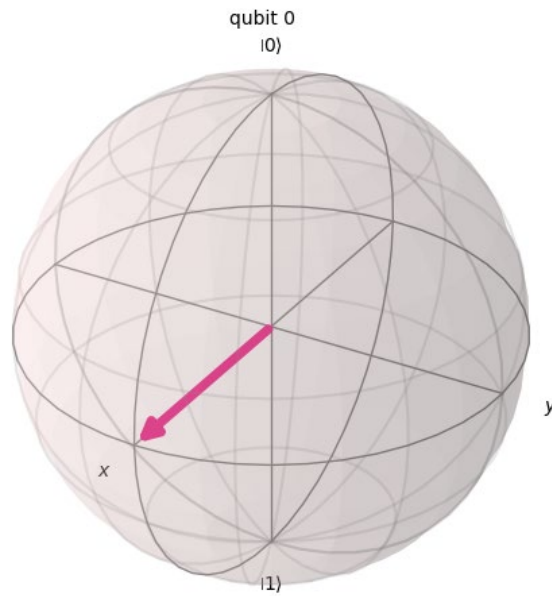
state = Statevector.from_instruction(qc)
print("Statevector:", state)

plot_bloch_multivector(state, title="Program 8: H then X")
```

OUTPUT:



Program 8: H then X



Program 9: Compare the Bloch Sphere representations obtained using Pauli-X, Pauli-Y, and Pauli-Z gates.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

# qubit0 -> X, qubit1 -> Y, qubit2 -> Z, compared side-by-side
qc = QuantumCircuit(3)
qc.x(0)
qc.y(1)
qc.z(2)
print(qc.draw())

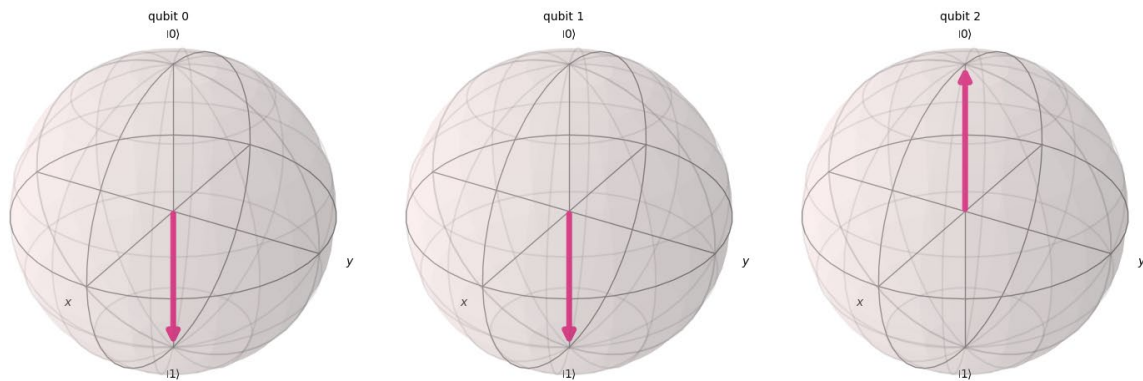
state = Statevector.from_instruction(qc)
print("Statevector:", state)

plot_bloch_multivector(state, title="Program 9: q0=X q1=Y q2=Z")
```

OUTPUT:



Program 9: $q_0=X$ $q_1=Y$ $q_2=Z$ (compared)



Program 10: Generate the state vector of a single-qubit quantum circuit and visualize its corresponding Bloch Sphere representation using Qiskit.

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

# General single-qubit circuit: superposition (H) + phase gate (T)
qc = QuantumCircuit(1)
qc.h(0)
qc.t(0)                                # T gate adds a pi/4 phase -> point on equator
print(qc.draw())

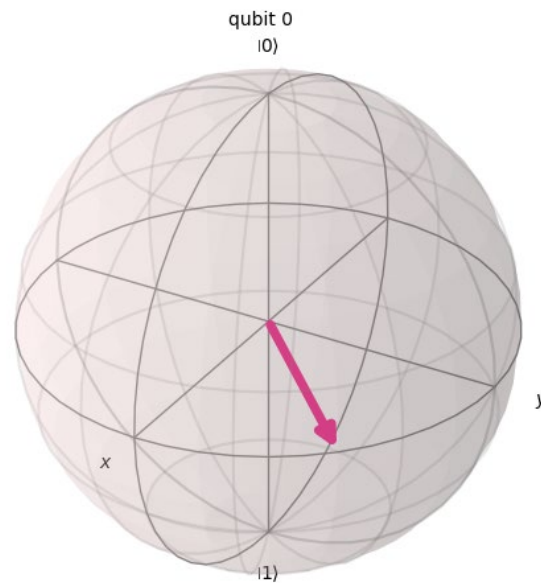
state = Statevector.from_instruction(qc)
print("Statevector:", state)
print("Probabilities:", state.probabilities_dict())

plot_bloch_multivector(state, title="Program 10: General state vector (H + T)")
```

OUTPUT:



Program 10: General state vector (H + T)



VIVA Q&A:

1. What is a Bloch Sphere?

Answer: The Bloch Sphere is a unit sphere used to geometrically represent the pure state of a single qubit, with every point on its surface corresponding to one possible qubit state.

2. Why is the Bloch Sphere used in quantum computing?

Answer: It gives an intuitive, visual way to represent qubit states and to see how quantum gates rotate or transform those states, making abstract complex-amplitude math easier to understand.

3. How is a single-qubit state represented on the Bloch Sphere?

Answer: It is represented using two angles, θ (polar angle) and ϕ (azimuthal angle), as $|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\phi} \sin(\theta/2)|1\rangle$, which fixes a unique point on the sphere's surface.

4. What is quantum superposition?

Answer: Superposition is the ability of a qubit to exist in a combination of both $|0\rangle$ and $|1\rangle$ basis states simultaneously, rather than being restricted to just one classical value.

5. What is the mathematical representation of a qubit?

Answer: A qubit is represented as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are complex probability amplitudes satisfying $|\alpha|^2 + |\beta|^2 = 1$.

6. What is the effect of the Hadamard gate on a qubit?



Answer: The Hadamard gate transforms $|0\rangle$ into the equal superposition state $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, moving the qubit from the pole to the equator of the Bloch Sphere.

7. What is the function of the Pauli-X gate?

Answer: The Pauli-X gate acts like a classical NOT gate — it flips $|0\rangle$ to $|1\rangle$ and vice versa, corresponding to a 180° rotation about the X-axis of the Bloch Sphere.

8. What is the difference between the Pauli-X, Pauli-Y, and Pauli-Z gates?

Answer: Pauli-X flips the qubit about the X-axis (bit-flip), Pauli-Y flips it about the Y-axis (combined bit- and phase-flip), and Pauli-Z flips it about the Z-axis (phase-flip only, leaving $|0\rangle$ and $|1\rangle$ populations unchanged).

9. Which Qiskit function is used to visualize a Bloch Sphere?

Answer: The `plot_bloch_multivector()` function (from `qiskit.visualization`), which takes a `Statevector/DensityMatrix` and plots the Bloch sphere for each qubit.

10. What is the significance of Bloch Sphere representation in quantum computing?

Answer: It provides a fundamental, intuitive visualization for understanding quantum state evolution, gate operations and superposition, forming the geometric backbone for reasoning about single-qubit quantum algorithms.

RESULT:

The representation of single-qubit quantum states using the Bloch Sphere was studied, and the effect of the Pauli-X, Pauli-Y, Pauli-Z and Hadamard gates on a qubit was successfully visualized using the Qiskit framework. The computational basis states $|0\rangle$ and $|1\rangle$ were observed at the poles, the superposition states $|+\rangle$ and $|-\rangle$ were observed on the equator, and the rotations produced by each quantum gate were verified on the Bloch Sphere. The experiment thus demonstrates a clear understanding of qubit state representation, superposition, and gate-induced state transformations.